

```

❸ submission_dicts = []
    for submission_id in submission_ids[:5]:
        # Make a new API call for each submission.
❹    url = f"https://hacker-news.firebaseio.com/v0/item/{submission_id}.json"
        r = requests.get(url)
        print(f"id: {submission_id}\tstatus: {r.status_code}")
        response_dict = r.json()

        # Build a dictionary for each article.
❺    submission_dict = {
        'title': response_dict['title'],
        'hn_link': f"https://news.ycombinator.com/item?id={submission_id}",
        'comments': response_dict['descendants'],
        }
❻    submission_dicts.append(submission_dict)

❼ submission_dicts = sorted(submission_dicts, key=itemgetter('comments'),
                            reverse=True)

❽ for submission_dict in submission_dicts:
    print(f"\nTitle: {submission_dict['title']}")
    print(f"Discussion link: {submission_dict['hn_link']}")
    print(f"Comments: {submission_dict['comments']}")

```

First, we make an API call and print the status of the response ❶. This API call returns a list containing the IDs of up to 500 of the most popular articles on Hacker News at the time the call is issued. We then convert the response object to a Python list ❷, which we assign to `submission_ids`. We'll use these IDs to build a set of dictionaries, each of which contains information about one of the current submissions.

We set up an empty list called `submission_dicts` to store these dictionaries ❸. We then loop through the IDs of the top 30 submissions. We make a new API call for each submission by generating a URL that includes the current value of `submission_id` ❹. We print the status of each request along with its ID, so we can see whether it's successful.

Next, we create a dictionary for the submission currently being processed ❺. We store the title of the submission, a link to the discussion page for that item, and the number of comments the article has received so far. Then we append each `submission_dict` to the list `submission_dicts` ❻.

Each submission on Hacker News is ranked according to an overall score based on a number of factors, including how many times it's been voted on, how many comments it's received, and how recent the submission is. We want to sort the list of dictionaries by the number of comments. To do this, we use a function called `itemgetter()` ❼, which comes from the operator module. We pass this function the key `'comments'`, and it pulls the value associated with that key from each dictionary in the list. The `sorted()` function then uses this value as its basis for sorting the list. We sort the list in reverse order, to place the most-commented stories first.

Once the list is sorted, we loop through the list ❽ and print out three pieces of information about each of the top submissions: the title, a link